

Programming Fundamentals

Lab (CL1002)

Date: December 18th 2025**Instructor(s)**

Ms. Marwa Khan, Mr. Muhammad Hasan, Mr. Umar Farooq
 Ms. Namra Absar, Mr. Ahmad Jawad, Ms. Reshail Fatima,
 Mr. Aqib Zeeshan, Ms. Seemab Ayub, Ms. Halima Sadia,
 Ms. Sadia Shafique, Ms. Anosha

Final Exam

Total Time (Hrs): 2.5
 Total Marks: 65
 Total Questions: 3

Important Instructions (Please read them before attempting the exam):

- Put your all .cpp files in single folder name it Roll_no_Section_Final(25L_1234) and zip that folder and name of the folder is same as mentioned.
- You are not allowed to use Internet or any Gadgets including USB device, cell phones
- Plagiarism will result in F grade in lab.
- Submission path: \\Exam\Fall 2025 Final Exam\Programming Fundamentals
- Make sure that the interface of your program is user friendly i.e. properly display information.
- No conversation allowed; violation will cancel your final exam
- Protect your code; if copied, you're responsible.
- No questions allowed; make assumptions and proceed.
- Properly follow the coding standards, correct formatting and indentation.
- Read questions carefully.
- Use Visual Studio 2013/2022.

Question 1:**[15 Marks]**

A meteorological research team has recorded daily average temperatures (in °C) for N days in a 1D array. Due to a sensor error, some entries were recorded as -999 (invalid data). Write a complete C++ program that:

- Read N (total number of elements) and N **temperature values** from the user.
- Removes all invalid entries (-999) from the array logically (without using vectors – shift valid elements left and track new logical size of the array).
- Calculates the longest consecutive sequence length where temperatures are strictly decreasing.
 - A sequence is strictly decreasing if $arr[i] < arr[i - 1]$
- Writes the cleaned array (valid temperatures only) and the longest decreasing consecutive sequence.
- Uses at least two functions:
 - `cleanData()`: takes original array and N, removes -999 values by left-shifting valid elements in the same array and returns the logical size (count of remaining valid elements).
 - `longestDecreasingSeq()`: takes cleaned array and its size, returns the length of the longest strictly decreasing consecutive sequence.

Function Prototypes

```
int cleanData(int arr[], int n);
int longestDecreasingSeq(int arr[], int n);
```

Example:

Fall 2025

Input: 50 -999 48 46 44 -999 43 41 39 37 35 33 $\Rightarrow 12$ Output: Cleaned Temperatures: 50 48 46 44 43 41 39 37 35 33 $\Rightarrow 10$ Longest Decreasing Sequence Length: 10
Input: 60 -999 58 57 59 -999 55 54 53 52 51 50 Output: Cleaned Temperatures: 60 58 57 59 55 54 53 52 51 50 Longest Decreasing Sequence Length: 7

Question 2:

[20 Marks]

You are given a character array that stores a paragraph consisting of multiple words separated by spaces. The text may contain repeated words. All characters are in lowercase alphabets. You are required to perform the following tasks using only basic programming constructs.

PART A — Removal of stop words Words [8 Marks]

Some commonly used words must be removed from the given text. The stop words are defined as:

`char stop_words[][6] = {"to", "in", "on", "for", "with"}.`

Your task is to remove all occurrences of these words from the text, while preserving the order of the remaining words. Ensure that no extra spaces appear in the modified text. The function prototype:

`void removeStopWords(char text[], char stop_words[][6], int size);`

Example:

Input:

`char text[] = "learning programming in fast for fast students with passion";`

Output:

learning programming fast fast students passion

PART B — Word Frequency Count [6 Marks]

Using the modified text obtained after Part A, display the frequency of each distinct word. The function prototype: `void display_word_frequency(char text[]);`

Example Output:

```
learning : 1
programming : 1
fast : 2
students : 1
passion : 1
```

PART C — Longest Word in Modified Text [6 Marks]

Using the **text obtained after removing stopwords** (from Part A), identify the **longest word**. Words are considered to be separated by spaces. If multiple words have the same maximum length, display the **first one**. The function prototype: `void find_longest_word(char text[]);`

Example Input (after stopword removal):

learning programming fast fast students passion

Example Output:

Longest word: programming

Important Instructions

Do not use STL containers or the string class, and built-in string handling functions are not allowed. You must use only character arrays, loops, and conditional statements to implement the required functionality. You may assume that the maximum length of the text will not exceed 500 characters.

Question 3:

[3+8+8+8+3= 40 Marks]

You are given a file `heat.txt` containing data for 24 hours $\times$ 20 sensors. Each value = temperature in $^{\circ}\text{C}$.

PART A: Detect Hourly Anomalies (8 marks)

For each hour (row), an hour is considered "anomalous" if it contains at least three increasing adjacent triplets (e.g., 12, 13, 14) and at least two values greater than 80°C . Store all anomalous hours in an array.

PART B: Sensor Stability Score (8 marks)

For each sensor (column), compute its Stability Score, defined as the number of times the sensor's reading is within ± 2 of the previous hour. For example, if the sensor values are {50, 52, 53, 70}, then $|52-50| \leq 2$ and $|53-52| \leq 2$ count toward the score, while $|70-53| > 2$ does not, giving a score of 2.

Determine the **highest stability score** along with the corresponding **sensor index**, and also count **how many sensors have a score ≥ 20** .

PART C: Heatmap Quadrant Dominance (8 marks)

Divide the 24×20 matrix into four quadrants: Q1 (rows 0–11, cols 0–9), Q2 (rows 0–11, cols 10–19), Q3 (rows 12–23, cols 0–9), and Q4 (rows 12–23, cols 10–19). Compute the average temperature of each quadrant and identify the dominant quadrant with the highest average.

PART D: Write all results to `analysis.txt` (3 marks)

Sample Input:

`heat.txt`:

Sample Output:

`analysis.txt`:

Anomalous hours count: 5

Anomalous hours: 3 4 5 8 12

Highest stability sensor:

0 with score 23

1 with score 23

2 with score 23

3 with score 23

Sensors with score ≥ 20 : 18

Quadrant averages:

Q1: 36.225

Q2: 47.05

Q3: 23.25

Q4: 44.09

Dominant quadrant: Q2